

StockRush

분산 커머스 / 주문 상태
수령 / 이벤트 기반 백엔드

담당 범위

개인 프로젝트 / 설계,
구현, 검증

주요 기술

Spring Boot

Kafka

PostgreSQL

Keycloak

결제 지연과 이벤트 중복 뒤에도 주문 상태를 수렴시키는 커머스

한정 재고 주문에서 결제 지연·이벤트 중복·Kafka 중단이 겹쳐도 주문을 취소·확정 상태로 수렴시키도록 Saga, Outbox, 멍등 소비자를 구현했습니다.

시스템 흐름

실패 조건과 보호 경계

Client
01Gateway
02Order Saga
03Inventory /
Payment
04Outbox /
Kafka
05Read Model
06

멍등 소비

동일 이벤트 2회 →
Outbox 1건

처리 완료 이벤트 키로 중복 차단

종료 상태 조건

늦은 승인 → 무시
취소 주문 상태 유지

발행 재시도

5회 실패 → FAILED

실패 상태와 재처리 경로 보존

해결한 문제

재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남

동시 주문, 결제 실패와 지연, Kafka 중단, 이벤트 재처리 상황에서는 주문 상태와 재고 수량, 결제 결과, 조회 모델이 서로 다른 속도로 바뀝니다. 서비스 경계와 복구 흐름을 코드와 시나리오로 분리해 최종 상태 수렴을 검증했습니다.

핵심 설계 판단

Saga 상태 조율

재고·결제·쿠폰·출고 상태 전이를 주문 서비스에서
조율

신뢰할 수 있는 이벤트 발행

DB 변경과 Outbox 행을 함께 저장해 발행 실패를
재처리

대표 코드 근거

PersistentCreateOrderService.create

파일 services/order-
service/src/main/java/com/stockrush/order/application/Persistent
CreateOrderService.java

```
boolean saved =
orderRepository.saveIfAbsent(result.order(),
command.idempotencyKey());
if (!saved) {
return
CreateOrderResult.replayed(replayOrder(command.id
empotencyKey(), command.memberId()));
}
outboxEventRepository.save(result.outboxEvent());
```

보호 규칙 주문 저장과 Outbox 기록을 같은 트랜잭션에 두고 멍등 충돌은
기존 주문 재조회로 수렴시킵니다.

연결 테스트 services/order-
service/src/test/java/com/stockrush/order/infra/persistence/P
ersistentCreateOrderServiceIntegrationTest.java
PersistentCreateOrderServiceIntegrationTest

구현 결과 ems_and_pending_outbox_event_together

동일 이벤트를 두 번 처리해도 Outbox가 1건만
남고, 취소 뒤 늦은 결제 승인은 주문 상태를 바꾸지
않았습니다.

검증 결과

- OrderSagaEventHandlerIntegrationTest
동일 이벤트 2회 처리에도 Outbox 1건
남도록 고정합니다.

- 취소 완료 뒤 늦게 도착한 결제 승인이
주문을 되살리지 않는 상태 guard를
검증합니다.

확정한 범위와 남은 경계

코드와 통합 테스트로 멍등 소비·종료 상태 조건·
발행 재시도를 다뤘으며 실패·다중 리전·운영 규모
부하는 포함하지 않았습니다.